TASK TELANGANA · CLOUD SOFTWARE DEVELOPER TRAINING

Module 03: Hands-on Lab Manual & Project Guide

Complete Step-by-Step Practical Playbook: EC2, Storage (EBS/S3), Custom VPC & Mini Project 1

Trainer: Sundeep Hiremat | **Center:** Nalgonda Training Center | **Curriculum:** AWS DVA-C02 Alignment

LAB INSTRUCTIONS FOR STUDENTS: Every lab in this manual is verified strictly within the AWS Free Tier. Follow all configuration parameters carefully (especially Availability Zones, Security Group rules, and IAM permissions). Before leaving the training room, always perform the **Sunset Cleanup Routine** to keep your billing at $0.00.

Lab 1: Amazon EC2 Launch, User Data Automation & Custom AMIs

Objective: Launch Amazon Linux 2023 EC2 instance on Free Tier (`t2.micro` or `t3.micro`), inject User Data startup automation script, verify live web response, and bake a Custom AMI Golden Image.

Step-by-Step Procedure:

1. Open **EC2 Dashboard** ➔ Click **Launch Instance**.
2. **Name:** task-webserver-01 | **AMI:** *Amazon Linux 2023 AMI (Free tier eligible)*.
3. **Instance type:** t2.micro or t3.micro | **Key pair:** Create task-nalgonda-key.pem (RSA format).
4. **Security Group:** Allow **SSH (Port 22, Source: My IP)** and **HTTP (Port 80, Source: 0.0.0.0/0)**.
5. **User Data (Advanced Details):** Paste the following bootstrapping script to auto-install Apache & Python on first boot:

```
#!/bin/bash
dnf update -y && dnf install -y httpd python3 git
systemctl start httpd && systemctl enable httpd
TOKEN=$(curl -s -X PUT 'http://169.254.169.254/latest/api/token' -H 'X-aws-ec2-metadata-token-ttl-seconds: 21600')
IID=$(curl -s -H 'X-aws-ec2-metadata-token: '$TOKEN http://169.254.169.254/latest/meta-data/instance-id)
echo '<h1>TASK Cloud Lab · Instance: '$IID'</h1>' > /var/www/html/index.html
```

6. Click **Launch instance** ➔ Wait until state is *Running* ➔ Open http://<PUBLIC-IP> in browser.
7. **Bake Custom AMI:** Select instance ➔ **Actions** ➔ **Image and templates** ➔ **Create image** ➔ Name: task-golden-v1 ➔ Click **Create image**.

Lab 2: Storage Triad — EBS Formatting & Secure S3 Buckets

Objective: Attach and format an EBS volume (`ext4`), create snapshots, provision an Amazon S3 bucket with Block Public Access, and enforce HTTPS via JSON Bucket Policy.

Part A: EBS Volume Mount & Snapshots

1. EC2 Console ➔ **Volumes** ➔ **Create volume** ➔ 5 GiB gp3 ➔ Select the **SAME Availability Zone** as your EC2 instance.
2. Select Volume ➔ **Actions** ➔ **Attach volume** ➔ Attach to task-webserver-01.
3. Connect to EC2 terminal and format the block storage:

```
lsblk # View attached block storage device (xvdf)
sudo mkfs -t ext4 /dev/xvdf # Format raw block storage with ext4 filesystem
sudo mkdir -p /mnt/data # Create mount directory
sudo mount /dev/xvdf /mnt/data # Mount volume
df -h /mnt/data # Verify available storage space
```

4. **EBS Snapshot:** In AWS Console ➔ Select Volume ➔ **Actions** ➔ **Create snapshot**.

Part B: Amazon S3 & HTTPS Bucket Policy

1. S3 Console ➔ **Create bucket** ➔ Name: task-nalgonda-<rollno>-2026 (globally unique).
2. Ensure **Block all public access** is CHECKED ➔ Click **Create bucket** ➔ Upload sample file.
3. Go to **Permissions** ➔ **Bucket policy** ➔ **Edit** ➔ Paste HTTPS enforcement JSON policy:

```
{ "Version": "2012-10-17", "Statement": [{ "Sid": "EnforceHTTPSOnly", "Effect": "Deny", "Principal": "*",
"Action": "s3:*", "Resource": ["arn:aws:s3:::YOUR-BUCKET-NAME", "arn:aws:s3:::YOUR-BUCKET-NAME/*"],
"Condition": {"Bool": {"aws:SecureTransport": "false"}} }] }
```

Lab 3: Custom Amazon VPC, Subnets, Gateways & Security

Objective: Build an isolated corporate VPC (`10.0.0.0/16`), carve Public and Private subnets, attach an Internet Gateway (IGW), configure Route Tables, and test Stateful Security Groups vs Stateless Network ACLs.

Component	CIDR / Configuration	Routing & Purpose
VPC	10.0.0.0/16	Isolated virtual network container (65,536 private IP addresses). Enable DNS hostnames.
Public Subnet	10.0.1.0/24 (AZ-1a)	Associated with Custom Route Table pointing 0.0.0.0/0 ➔ **Internet Gateway (IGW)**.
Private Subnet	10.0.2.0/24 (AZ-1b)	Associated with Main Route Table (Local only: 10.0.0.0/16). No direct route to IGW!
Security Group	**Stateful** (Instance Level)	Allow Inbound 80 HTTP; return traffic is automatically permitted out.
Network ACL	**Stateless** (Subnet Level)	Must explicitly permit BOTH Inbound (port 80) and Outbound (ports 1024–65535).

Lab 4 (Mini Project 1): Live Portfolio on Ubuntu 24.04 LTS + Nginx + GitHub

Objective: Launch Ubuntu 24.04 LTS, install Git and Nginx, clone personal portfolio from GitHub into /var/www/html, fix Linux permissions, and verify live web deployment.

```
# 1. Update OS package lists and upgrade existing software
sudo apt update && sudo apt upgrade -y

# 2. Install Git and high-performance Nginx web server
sudo apt install git nginx -y
sudo systemctl status nginx # Verify: Active (running)

# 3. Remove default Nginx welcome placeholder
cd /var/www/html && sudo rm -f index.nginx-debian.html

# 4. Clone your GitHub portfolio into web root
sudo git clone https://github.com/gauri-abc/gauri-yadav-cloud-portfolio.git .

# 5. Configure Linux ownership (www-data) and permissions (755)
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 755 /var/www/html

# 6. Restart & enable Nginx service
sudo systemctl restart nginx && sudo systemctl enable nginx
```

7. Open browser: http://<YOUR_EC2_PUBLIC_IP> to verify live portfolio!

Student Zero-Dollar Sunset Checklist

1. Stop practice EC2 instances (pauses compute charges).
2. Terminate throwaway instances and delete orphan EBS volumes.
3. Release any allocated Elastic IPs.
4. Delete practice CloudFormation stacks to tear down child resources cleanly.
5. Check AWS Billing Dashboard weekly to ensure $0.00 balance.